SECURITY AUDIT OF

OP_NET AS-BigNum

Public Report

Version: 1.1

March 13, 2026

Verichains Lab

info@verichains.io

https://www.verichains.io

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **March 13, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the OP_NET As-Bignum. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit and penetration testing process, the audit team had identified some issues in the source code.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY 4
1.1. About OP_NET AS-BigNum 4
1.2. Audit Methodology 4
1.3. Disclaimer 4
1.4. Acceptance Minute 5
2. AUDIT OVERVIEW 6
2.1. Audit Scope 6
2.2. Overview 6
3. FINDINGS 8
3.1. `H-01` - Inadequate Overflow Detection in `u128Safe.add()` `[HIGH]` `[FIXED]` 8
3.2. `H-02` - Incorrect Remainder in `u128.rem10()` `[HIGH]` `[FIXED]` 9
3.3. `H-03` - Faulty Comparison Logic in `u128.ord()` `[HIGH]` `[FIXED]` 11
3.4. `H-04` - Overflow Issues in `u128.muldiv()` `[HIGH]` `[FIXED]` 12
3.5. `H-05` - Incorrect Squaring in `u128.sqr()` `[HIGH]` `[FIXED]` 14
3.6. `L-01` - Wrong logic in `i256.isEmpty()` `[LOW]` `[FIXED]` 15
4. VERSION HISTORY 16

1. MANAGEMENT SUMMARY

1.1. About OP_NET AS-BigNum

OP_NET AS-BigNum provides wide numeric types — `u128`, `u256`, `i128`, and `i256` — with full arithmetic operations support. It's designed for blockchain smart contracts, cryptographic operations, and any use case requiring deterministic behavior with large integers in WebAssembly environments.

1.2. Audit Methodology

Our security audit process for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using public and our in-house smart contract security analysis tool.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that were considered during the audit of the smart contract:

- Integer Overflow and Underflow
- Timestamp Dependence
- Race Conditions
- Transaction-Ordering Dependence
- DoS with (Unexpected) revert
- DoS with Block Gas Limit
- Gas Usage, Gas Limit and Loops
- Redundant fallback function
- Unsafe type Inference
- Reentrancy
- Explicit visibility of functions state variables (external, internal, private and public)
- Logic Flaws

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and

shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

The audit focus on identifying security flaws in the code of OP_NET AS-BigNum. It was first conducted on commit `0ce1d0f2d002082bf5eae2ed64cba6e358eaf36d` from git repository `https://github.com/btc-vision/as-bignum`. Latest audited commit is `5156d8aaeb1d4db5d0b9fbb885b1626be65a331d`.

This table summarizes the report revision history, mapping each report version to its corresponding Git commit hash.

Report Version	Git Commit
0.1	`0ce1d0f2d002082bf5eae2ed64cba6e358eaf36d`
0.2	`5313a4044f58073a5f421d1e3ae880d187ae08d9`
1.0	`5156d8aaeb1d4db5d0b9fbb885b1626be65a331d`
1.1	`5156d8aaeb1d4db5d0b9fbb885b1626be65a331d`

2.2. Overview

This library provides arbitrary-precision integer arithmetic types for the OP_NET smart contracts. These types extend beyond native WebAssembly's 64-bit integers, enabling large number operations for cryptographic functions and token balances, as well as precise decimal arithmetic for financial calculations.

2.2.1. Integer Module

The integer module delivers a robust, high-performance implementation of 128-bit and 256-bit integers, meticulously optimized for the gas-metered constraints of the OP_NET virtual machine. At the core of this module are the `u128` and `u256` classes, which define unsigned integer types constructed from multiple 64-bit limbs (`lo`/`hi` for 128-bit, and `lo1` through `hi2` for 256-bit). These classes are not merely storage containers but feature comprehensive support for a full range of arithmetic operations—including addition, subtraction, multiplication, division, and modulo—as well as extensive bitwise logic (AND, OR, XOR, NOT), shifting, and rotations. To ensure high performance and low execution costs, critical operations minimize control flow overhead by utilizing unrolled loops, direct memory access for byte transformations, and specialized optimization techniques such as immediate carry propagation.

To strictly enforce gas efficiency, the module relies heavily on `globals.ts`, a collection of highly optimized global helper functions. Complex operations that would otherwise be computationally expensive in pure high-level logic, such as 256-bit multiplication or division, are offloaded to these specialized helpers like `__multi3` and `__u256carry`. These helpers are tuned to the specific cost model of the OP_NET virtual machine, ensuring that common large-integer operations consume minimal gas resources. For example, multiplication algorithms leverage Karatsuba-like decomposition or straightforward limb-based math depending on the bit width to balance opcode count against stack usage.

Complementing the unsigned types are `i128` and `i256`, which implement signed integers using standard two's complement representation. The `i128` class offers full signed arithmetic capabilities, automatically handling sign extension and overflow conditions, along with utilities for sign checks (`isPos`, `isNeg`) and absolute value calculation. The `i256` class currently provides the structural basis for 256-bit signed values, intended for future expansion. All integer types are exported via `index.ts` and include robust utilities from `utils.ts` for string parsing and formatting in various radices (decimal, hex, etc.), as well as seamless conversion to and from byte arrays in both little-endian and big-endian formats.

2.2.2. Data Flow

The system uses a hierarchical dependency structure where higher-level signed types rely on their unsigned counterparts for underlying storage and bitwise operations, ensuring consistency and reusability. Both `u128` and `u256` delegate their most complex or potentially expensive calculations to the optimized global functions found in `globals.ts`. This delegation strategy allows the core classes to remain lightweight and readable while pushing the heavy computational lifting to optimized routines.

For instance, when a contract calls `u256.add(a, b)`, the operation does not simply run a loop; it triggers `__u256carry`, a specialized function that efficiently handles arithmetic across the four 64-bit limbs using tailored instructions to propagate carries without unnecessary branching. Similarly, division operations utilize `__divmod` helpers that simplify the process into manageable 64-bit steps. This architectural separation ensures that the VM's gas metering middleware can accurately track execution costs at a granular level, while simultaneously guaranteeing that all arithmetic results remain strictly deterministic across all supported integer widths, specifically avoiding the non-determinism associated with floating-point math.

3. FINDINGS

3.1. `H-01` - Inadequate Overflow Detection in `u128Safe.add()` `[HIGH]` `FIXED`

SCOPE

- **Commit**: `c6772099afbe93b97ea3d793e64fb7cb5478c349`
- **Files**: `assembly/integer/safe/u128.ts`

DESCRIPTION

```
if (((hi ^ x) & (hi ^ y)) < c)
```

fails to reliably detect all overflow scenarios. As a result, the following testcase - which expects an exception on overflow - fails:

```
it("test safe add", () => {
    expect(() => {
        let a = new u128Safe(14578515230910040473, 11834292359786415467);
        let b = new u128Safe(9202606200622059783, 8690157821842247249);
        !(a + b);
    }).toThrow();
});
```

UPDATE

The safe implement of u128 had been removed.

3.2. H-02 - Incorrect Remainder in u128.rem10() [HIGH] FIXED

SCOPE

- **Commit**: c6772099afbe93b97ea3d793e64fb7cb5478c349
- **Files**:
  - assembly/integer/u128.ts
  - assembly/globals.ts

DESCRIPTION

File: assembly/integer/u128.ts

```
    @inline
    static rem10(value: u128): u128 {
        __udivmod128_10(value.lo, value.hi);
        return new u128(__divmod_rem_lo, __divmod_rem_hi);
    }
```

File: assembly/globals.ts

```
540:    @global
541:    export function __udivmod128_10(lo: u64, hi: u64): u64 {
542:        if (!hi) {
543:            __divmod_quot_hi = 0;
544:            if (lo < 10) {
545:                __divmod_rem_lo = 0;
546:                __divmod_rem_hi = 0;
547:                return 0;
548:            } else {
549:                let qlo = lo / 10;
550:                __divmod_rem_lo = lo - qlo * 10;
551:                __divmod_rem_hi = 0;
552:                return qlo;
553:            }
554:        }
555:
556:    var q: u128, r: u128;
557:    var n = new u128(lo, hi);
558:
559:    q = n >> 1;
560:    q += n >> 2;
561:    q += q >> 4;
562:    q += q >> 8;
563:    q += q >> 16;
564:    q += q >> 32;
565:    q += u128.fromU64(q.hi); // q >> 64
566:    q >>= 3;
567:    r = n - (((q << 2) + q) << 1);
568:    n = q + u128.fromBool(r.lo > 9);
569:
570:    __divmod_quot_hi = n.hi;
571:    __divmod_rem_lo = r.lo;
572:    __divmod_rem_hi = r.hi;
573:    return n.lo;
574: }
```

The u128.rem10() method relies on the helper function __udivmod128_10() (defined in globals.ts) to compute the remainder when dividing by 10. Two issues in __udivmod128_10() cause erroneous results:

- **Faulty Handling for Small Values:**

When the input are less than 10 (line 545), the function explicitly sets __divmod_rem_lo to 0 . This results in rem10() returning 0 for any u128 value below 10 instead of returning the original value. For example, the testcase:

```
it("test rem10 1", () => {
    let a = new u128(2, 0);
    expect(u128.rem10(a)).toStrictEqual(new u128(2, 0));
});
```

fails due to this misbehavior.

- **Incorrect Quotient Adjustment Without Compensating Remainder:**

At line 568, if the computed remainder `r` exceeds 9, the quotient is incremented by one. However, the remainder is not adjusted (i.e., decreased) correspondingly, leading to an imbalance in the division result. This defect is demonstrated by:

```
it("test rem10 2", () => {
    let a = new u128(16222495557571259006, 18092382526624944601);
    expect(u128.rem10(a)).toStrictEqual(new u128(2, 0));
});
```

UPDATE

The issue had been fixed in commit `b49c85237355320b7b033b7401f0fd1be703e6dd` .

3.3. `H-03` - Faulty Comparison Logic in `u128.ord()` `[HIGH]` `FIXED`

SCOPE

- **Commit**: `c6772099afbe93b97ea3d793e64fb7cb5478c349`
- **Files**: `assembly/integer/u128.ts`

DESCRIPTION

```
531: /**
532: * Get ordering
533: * if a > b then result is 1
534: * if a < b then result is -1
535: * if a = b then result is 0
536: * @param a 128-bit unsigned integer
537: * @param b 128-bit unsigned integer
538: * @returns 32-bit signed integer
539: */
540:    @inline
541:    static ord(a: u128, b: u128): i32 {
542:        var dlo = a.lo - b.lo;
543:        var dhi = a.hi - b.hi;
544:        var cmp = <i32>select<i64>(dhi, dlo, dhi != 0);
545:        // normalize to [-1, 0, 1]
546:        return i32(cmp > 0) - i32(cmp < 0);
547:    }
```

The implementation (specifically the selection logic at line 544) is incorrect. As a result, in certain comparisons the wrong result is produced. For example:

```
it("test ord", () => {
    let a = new u128(11862732932339670732, 0);
    let b = new u128(11791870301324684439, 10039605328933046698);
    expect(u128.ord(a, b)).toBe(-1);
});
```

UPDATE

The function had been changed into:

```
/**
 * Get ordering
 * if a > b then result is  1
 * if a < b then result is -1
 * if a = b then result is  0
 * @param  a 128-bit unsigned integer
 * @param  b 128-bit unsigned integer
 * @returns  32-bit signed integer
 */
@inline
static ord(a: u128, b: u128): i32 {
  // if a > b => +1
  // if a < b => -1
  // if equal => 0
  if (a.hi > b.hi) return 1;
  if (a.hi < b.hi) return -1;
  if (a.lo > b.lo) return 1;
  if (a.lo < b.lo) return -1;
  return 0;
}
```

3.4. H-04 - Overflow Issues in u128.muldiv() [HIGH] FIXED

SCOPE

- **Commit**: c6772099afbe93b97ea3d793e64fb7cb5478c349
- **Files**: assembly/integer/u128.ts

DESCRIPTION

```
/**
 * Calculate multiply and division as `number * numerator / denominator`
 * without overflow in multiplication part.
 *
 * @returns 128-bit unsigned integer
 */
static muldiv(number: u128, numerator: u128, denominator: u128): u128 {
  let a = number;
  let b = numerator;
  let c = denominator;

  let ql = __udivmod128(b.lo, b.hi, c.lo, c.hi);

  let qn = new u128(ql, __divmod_quot_hi);             // b / c
  let rn = new u128(__divmod_rem_lo, __divmod_rem_hi); // b % c

  let q = u128.Zero;
  let r = u128.Zero;
  let n = a.clone();

  while (!n.isZero()) {
    if (n.lo & 1) {
      // @ts-ignore
      q += qn;
      // @ts-ignore
      r += rn;
      if (r >= c) {
        // @ts-ignore
        ++q;
        // @ts-ignore
        r -= c;
      }
    }
    // @ts-ignore
    n >>= 1;
    // @ts-ignore
    qn <<= 1;
    // @ts-ignore
    rn <<= 1;

    if (rn >= c) {
      // @ts-ignore
      ++qn;
      // @ts-ignore
      rn -= c;
    }
  }
  return q;
}
```

The function u128.muldiv() computes the operation (number * numerator) / denominator while attempting to avoid overflow in the multiplication phase. However, two vulnerabilities compromise its correctness:

- During the iterative process (line 614), the addition r += rn may overflow, invalidating the subsequent check if (r ⩾ c) .
- Similarly, the operation rn ⟸ 1 (line 627) is subject to overflow, leading to further inaccuracies.

A representative failing testcase is:

```
it("test muldiv", () => {
let a = new u128(15168594103972142211, 13112993984347045001);
let b = new u128(3336517327, 0);
```

```
let c = new u128(2932175525583687935, 10692182455030519441);
expect(u128.muldiv(a, b, c)).toStrictEqual(new u128(4091936498, 0));
});
```

UPDATE

The team switched from a self-implemented solution to using the `u128.mul` and `u128.div` .

3.5. H-05 - Incorrect Squaring in u128.sqr() [HIGH] FIXED

SCOPE

- **Commit**: c6772099afbe93b97ea3d793e64fb7cb5478c349
- **Files**: assembly/integer/u128.ts

DESCRIPTION

```
/**
   * Calculate inplace squared 128-bit unsigned integer (this ** 2)
   * @returns 128-bit unsigned integer
   */
  sqr(): this {
    var u = this.lo,
      v = this.hi;

    var u1 = u & 0xFFFFFFFF;
    var t = u1 * u1;
    var w = t & 0xFFFFFFFF;
    var k = t >> 32;

    u >>= 32;
    var m = u * u1;
    t = m + k;
    var w1 = t >> 32;

    t = m + (t & 0xFFFFFFFF);

    var lo = (t << 32) + w;
    var hi = u * u;
    hi += w1 + (t >> 32);
    hi += u * v << 1;

    this.lo = lo;
    this.hi = hi;

    return this;
  }
```

The u128.sqr() method computes the square of a 128-bit unsigned integer. An error is present at line 748, where the operation

```
hi += u * v << 1;
```

uses a modified version of the variable u (which was shifted at line 738) instead of the original lower half. The correct computation should use the original values (for instance, employing this.lo and this.hi) to correctly calculate the cross-product doubled. This mistake leads to a wrong squared result, as shown by:

```
it("test sqr", () => {
    let a = new u128(17448461305379225827, 14768026361276340486);
        expect((u128.sqr(a))).toStrictEqual(new u128(12113794852041967945, 18342544934353289530));
});
```

UPDATE

This function's return value was changed to u128.mul(this, this); . The team switched from a self-implemented solution to using the u128.mul function for sqr .

3.6. `L-01` - Wrong logic in `i256.isEmpty()` `[LOW]` `FIXED`

SCOPE

- Commit: `e9ca12affa0608f9dd7be724f44e8a5aef271f2a`
- File: `assembly/integer/i256.ts`

DESCRIPTION

```
export class i256 {
  @inline @operator.prefix('!')
  static isEmpty(value: i256): bool {
    return !value.isZero();
  }

  @inline
  isZero(): bool {
    return !(this.lo1 | this.lo2 | this.hi1 | this.hi2);
  }
}
```

The `isEmpty()` function contains inverted logic, returning `true` for **non-zero values** instead of zero values, which contradicts its intended semantics.

UPDATES

This issue was fixed in commit `5156d8aaeb1d4db5d0b9fbb885b1626be65a331d` .

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
1.0	*Jan 20, 2026*	Private Report	Verichains Lab
1.1	*Mar 13, 2026*	Public Report	Verichains Lab